

Ներածություն

Հայաստանում ավտոմեքենաների առքուվաճառքի շուկան դինամիկ զարգացող ոլորտներից է, սակայն դրան սպասարկող թվային ենթակառուցվածքները զգալիորեն հետ են մնում ժամանակակից վեբ տեխնոլոգիաների պահանջներից և անվտանգության ստանդարտներից: Ներկայումս շուկայում գործող պլատֆորմները, մասնավորապես՝ գերիշխող դիրք զբաղեցնող Auto.am-ը՝ կամ ընդհանուր նշանակության հայտարարությունների կայքերը (օրինակ՝ List.am), չեն ապահովում նեղ մասնագիտացված, անվտանգ և օգտատիրոջ համար հարմար (user-friendly) միջավայր:

Գործող համակարգերի հիմնական թերություններն են.

- Հնացած տեխնոլոգիա և ինտերֆեյս (UI/UX): Կայքերը տեխնիկապես հնացած են, ծանրաբեռնված են մեծաքանակ գովազդային վահանակներով և ունեն ոչ ինտուիտիվ դիզայն, ինչը բարդացնում է նպատակային տեղեկատվության որոնումը:
- Անվտանգության և վստահության բացակայություն: Վերիֆիկացիայի գործիքների և օգտատերերի օբյեկտիվ գնահատման (rating) համակարգի բացակայությունը անհնար է դարձնում բարեխիղճ օգտատերերի և զեղծարարների տարբերակումը՝ մեծացնելով ֆինանսական ռիսկերը:
- Ֆունկցիոնալ սահմանափակումներ: Հարթակները գործում են բացառապես որպես սովորական լիսթինգ հարթակներ՝ չտրամադրելով հաճախորդ-վաճառող կապի ապահովման ժամանակակից գործիքներ

Աշխատանքի նպատակը և առաջարկվող լուծումը

Ավարտական աշխատանքի նպատակն է նախագծել և պատրաստել ավտոմեքենաների առքուվաճառքի ժամանակակից, լիարժեք ֆունկցիոնալ վեբ հավելված, որը կլուծի վերոնշյալ խնդիրները՝ ապահովելով հուսալի հաճախորդ-սերվեր փոխազդեցություն, տվյալների բազայի օպտիմալ կառավարում և անվտանգ գործարքային միջավայր:

Հավելվածի հիմնական ֆունկցիոնալությունը

Հավելվածը հիմնված է դերային մոդելի վրա (գնորդ/վաճառող հստակ տարանջատմամբ) և տրամադրում է հետևյալ գործիքակազմը.

- Տվյալների մշակում և որոնում: Ներդրված է մեքենաների որոնման և բազմամակարդակ ֆիլտրացիայի համակարգ, որը հնարավորություն է տալիս արագ և ճշգրիտ դասակարգել ապրանքներն ըստ տեխնիկական և գնային պարամետրերի:
- Վաճառողի անհատական վահանակ (Dashboard): Պատրաստվել է ավտոմեքենաների հայտարարությունների մուտքագրման օպտիմիզացված և հարմարավետ ինտերֆեյս, որը երաշխավորում է բոլոր անհրաժեշտ տեխնիկական տվյալների ճշգրիտ հավաքագրումը:
- Գործարքների և հարցումների կառավարում: Համակարգն ապահովում է պոտենցիալ գնորդների ցանկի ցուցադրում, հայտերի մոնիտորինգ, ինչպես նաև վաճառողի կողմից առաջարկների ընդունման կամ մերժման ֆունկցիոնալություն:
- Գնորդի հայտերի վահանակ, որտեղ ցուցադրվում են ուղարկված հայտերը և վաճառողների պատասխանները:
- Ավտոմեքենաների հայտարարությունների համար առանձին էջեր, որտեղ գնորդները կարող են ավելի մանրամասն ծանոթանալ տվյալներին և ուղարկել հայտերը:

Գլուխ 1: Տեխնոլոգիաներ և ֆունկցիոնալություն

1.1: Օգտագործված տեխնոլոգիաները

Ծրագիրը պատրաստվել է ժամանակակից Full-Stack ճարտարապետությամբ, որն ապահովում է հաճախորդի (frontend), սերվերի (backend) և տվյալների բազայի անխափան փոխազդեցությունը: Այս մոտեցումը թույլ է տալիս ապահովել համակարգի մասշտաբելիությունը, հեշտ սպասարկումը և ֆունկցիոնալության ընդլայնման հնարավորությունը՝ առանց ամբողջ համակարգի վերափոխման: Բացի այդ, կիրառվել են մոդուլային կառուցվածքի և շերտավորված ճարտարապետության սկզբունքներ, որոնք բարձրացնում են կոդի ընթերցելիությունը, վերօգտագործելիությունը և ընդհանուր կայունությունը:

Հաճախորդի մակարդակ

Որպես ինտերֆեյսի մշակման հիմնական գործիքակազմ կիրառվել են TypeScript ծրագրավորման լեզուն և React.js գրադարանը: Այս համադրությունն ապահովում է բարձր արտադրողականություն ունեցող, դինամիկ և արագ արձագանքող միջավայր: Ի տարբերություն JavaScript-ի, TypeScript-ի միջոցով տիպերի խիստ ստուգումը զգալիորեն հեշտացնում է համակարգի սպասարկումը, բարձրացնում կոդի կանխատեսելիությունը և նվազեցնում աշխատանքի ընթացքում առաջացող սխալների հավանականությունը: React.js-ի կոմպոնենտային կառուցվածքը հնարավորություն է տալիս ստեղծել վերօգտագործելի և լավ կազմակերպված UI տարրեր, ինչը նպաստում է նախագծի մասշտաբելիությանը և հետագա զարգացման հեշտությանը: Բացի այդ, կիրառվել են ժամանակակից state կառավարման և API-ների հետ փոխազդեցության մեխանիզմներ, որոնք ապահովում են տվյալների արագ բեռնում և ինտերֆեյսի սահուն աշխատանքը:

Սերվերային մակարդակ և տվյալների բազա

Սերվերային տրամաբանության սպասարկման համար ընտրվել է Node.js միջավայրը, որը հիմնված է իրադարձահեն (event-driven) և ոչ բլոկավորող (non-blocking) ճարտարապետության վրա՝ ապահովելով բարձր արդյունավետություն և մեծ թվով հարցումների միաժամանակյա մշակման հնարավորություն: Backend հատվածում իրականացվել են RESTful API-ներ, որոնք ապահովում են հաճախորդի և սերվերի միջև հստակ կառուցված հաղորդակցություն: Տվյալների պահպանման և կառավարման նպատակով ինտեգրվել է MongoDB ոչ ռելյացիոն (NoSQL) բազան, որը թույլ է տալիս պահպանել տվյալները ձևով JSON-նման կառուցվածքով: Այս մոտեցումը հատկապես արդյունավետ է մեքենաների բազմազան և փոփոխական տեխնիկական պարամետրերի մշակման դեպքում, քանի որ տվյալների կառուցվածքը կարելի է հեշտորեն փոփոխել առանց բազայի ամբողջական վերակառուցման: Բացի այդ, կիրառվել են տվյալների վավերացման, անվտանգության (օրինակ՝ authentication և authorization մեխանիզմներ) և սխալների մշակման մեթոդներ, որոնք ապահովում են համակարգի հուսալի և անվտանգ աշխատանքը:

1.2: Հիմնական ֆունկցիոնալ հնարավորությունները

Նախագծի շրջանակներում մշակվել և ներդրվել են հետևյալ հնարավորությունները

1. Համակարգը խիստ տարանջատում է գնորդների և վաճառողների իրավունքները: Յուրաքանչյուր գործողության համար կա առանձնացված կառավարման վահանակ և գործողությունների եզակի թույլտվություններ, ինչը բացառում է համակարգում ոչ լիազորված միջամտությունները:
2. Վաճառողներին տրամադրվում է տրանսպորտային միջոցների հայտարարությունների ամբողջական կենսացիկլի կառավարման գործիքակազմ՝ ստեղծում, խմբագրում, ժամանակավոր կասեցում և հեռացում: Տվյալների բազայում պահպանվում են ոչ միայն ստանդարտ բնութագրերն ու լուսանկարները, այլև մեքենայի և դրա հրապարակումների պատմությունը:
3. Ներդրվել է դինամիկ որոնման մոդուլ, որը հնարավորություն է տալիս արդյունքները դասավորել խիստ մասնագիտացված պարամետրերով (գնային միջակայք, մակնիշ/մոդել, վառելիքի տեսակ, փոխանցման տուփ, վազք և այլն): Արդյունքների ցուցադրումն իրականացվում է էջավորման և տեսակավորման մեխանիզմներով՝ սերվերի օպտիմալ ծանրաբեռնվածությունն ապահովելու համար:
4. Գնման և վաճառքի գործընթացները ստանդարտացված են: Համակարգը թույլ է տալիս ուղարկել և ստանալ առաջարկներ, վերահսկել դրանց ընթացքը հստակ կարգավիճակներով («Սպասվող», «Ընդունված», «Մերժված», «Ավարտված»):
5. Մեղիա ֆայլերի (լուսանկարների) բեռնումը և մշակումը նախագծված են CDN (Content Delivery Network) ինտեգրացիայի պատրաստակամությամբ: Ռեսուրսատար պրոցեսները նախագծված են որպես ֆոնային աշխատանքներ (Background jobs), ինչը թույլ է տալիս չդանդաղեցնել հավելվածի հիմնական գործընթացի (main thread) աշխատանքը:

Գլուխ 2: Տվյալների բազաներ

2.1: Տվյալների բազաների կառավարման համակարգեր (DBMS)

Տվյալների բազաները, տվյալների որոշակի ձևով կազմակերպված համախումբ է, որում տվյալները պահպանվում են համաձայն նախապես որոշված նշանակման:

Ավանդական թղթային տվյալների պահոցներն ունեն մի շարք թերություններ, օրինակ՝

- պահանջվում են ֆիզիկական մեծ տարածքներ,
- անհրաժեշտ տվյալները գտնելու համար պահանջվում է բավականաչափ երկար ժամանակ,
- դարակները և կարգացանկերը կարգավորված վիճակում պահպանելու համար պետք է ծավալուն աշխատանք կատարել և մեծ ջանքեր գործադրել:

Տվյալների բազաները թույլ են տալիս լուծել բոլոր այդպիսի խնդիրները: Առավել լայն կիրառություն են գտել տվյալների ռեյացիոն և դոկումենտ-կոդմոնոքված բազաները: Ռեյացիոն բազաներում տեղեկատվությունը բաժանված է տրամաբանորեն կապակցված համեմատաբար փոքր և, դրա հետևանքով, ավելի հեշտ ղեկավարվող աղյուսակների: Աղյուսակները իրենց կազմակերպման մակարդակի շնորհիվ պարզեցնում են բազաներում ուղեկցումը և ապահովում դրա առավել նախընտրելի գործարկումը:

Տվյալների բազաների կառավարման համակարգը (ՏԲԿՀ) մասնագիտացված ծրագրային ապահովում է, որը նախատեսված է տվյալների կառուցվածքավորված պահպանման, որոնման, փոփոխման և անվտանգության ապահովման համար: Ի տարբերություն ֆայլային պարզ համակարգերի, ՏԲԿՀ-ն ապահովում է տվյալների ամբողջականությունը (integrity), բազմաօգտատեր հասանելիությունը և արդյունավետ մանիպուլյացիան նույնիսկ մեծածավալ տեղեկատվության պարագայում: Ժամանակակից վեբ մշակման մեջ տվյալների բազաները հիմնականում բաժանվում են երկու մեծ խմբի՝ SQL (Ռեյացիոն) և NoSQL (Ոչ ռեյացիոն):

Բացի այդ, տվյալների բազաների ընտրությունը պայմանավորված է կիրառման բնույթով և համակարգի պահանջներով. օրինակ՝ ռեյացիոն բազաները առավել հարմար են այն դեպքերում, երբ անհրաժեշտ է խիստ կառուցվածք, հստակ կապեր տվյալների միջև և բարդ հարցումներ, մինչդեռ NoSQL բազաները ավելի ճկուն են և արդյունավետ մեծածավալ կամ արագ փոփոխվող տվյալների մշակման համար: Այսպիսով, ճիշտ ընտրված տվյալների բազան և համապատասխան ՏԲԿՀ-ն էական դեր են խաղում համակարգի արտադրողականության, մասշտաբելիության և հուսալիության ապահովման գործում:

2.2: SQL (Ռեյացիոն տվյալների բազաներ)

Ռեյացիոն է կոչվում այն տվյալների բազան, որը բաղկացած է տրամաբանորեն փոխկապակցված աղյուսակներից: Աղյուսակը բազայում տվյալների պահպանման հիմնական միավորն է: Ռեյացիոն բազաներում ղեկավարման հատուկ համակարգի միջոցով ապահովվում են տվյալների հետ տարբեր գործառնությունների կատարման հնարավորություններ:

Ռեյացիոն բազաներում աղյուսակների տրամաբանական փոխկապակցվածությունը (ինչպես սովորաբար ասում են՝ աղյուսակները գտնվում են հարաբերությունների (relations) մեջ և այդ պատճառով կոչվում են աղյուսակ-հարաբերություններ) թույլ է տալիս մեկ հարցման միջոցով գտնել բոլոր անհրաժեշտ տվյալները, որոնք կարող են պարունակվել մի քանի աղյուսակներում:

Աղյուսակների միջև կապը իրականացվում է հիմնական (primary key) և արտաքին բանալիով (foreign key), այն դաշտը որ տվյալ աղյուսակի համար հիմնական բանալի է, դառնում է արտաքին բանալի այն աղյուսակի համար ում հետ կապված է: Օրինակ եթե ունենք ապրանք և պատվեր աղյուսակներ, կապը ապրանքի և պատվերի կարող ենք ցույց տալ հետևյալ կերպ: Ապրանքի աղյուսակում կստեղծվի յուրահատուկ դաշտ id, իսկ պատվեր աղյուսակում կստեղծվի product_id : product_id դաշտը ցույց կտա ապրանքի id ին:

Ռեյացիոն տվյալների բազաները հիմնված են մաթեմատիկական ռեյացիոն մոդելի վրա, որտեղ տվյալները կազմակերպվում են խիստ սահմանված աղյուսակներում:

- Տվյալները պահվում են տողերի և սյուների տեսքով: Յուրաքանչյուր աղյուսակ ունի կանխորոշված սխեմա, որը սահմանում է դաշտերի տեսակները և սահմանափակումները:
- SQL բազաները հետևում են ACID (Atomicity, Consistency, Isolation, Durability) պահանջներին, ինչը երաշխավորում է տրանզակցիաների բացարձակ հուսալիությունը: Սա կրիտիկական է ֆինանսական և բանկային համակարգերի համար:
- Տվյալների հետ փոխազդեցությունը կատարվում է SQL (Structured Query Language) լեզվի միջոցով, որը թույլ է տալիս կատարել բարդ հարցումներ և միավորել տարբեր աղյուսակներ (Joins):
- Հիմնականում մասշտաբավորվում են ուղղահայաց եղանակով (Vertical Scaling)՝ ավելացնելով գոյություն ունեցող սերվերի հզորությունը (CPU, RAM): Օրինակներ՝ PostgreSQL, MySQL, Microsoft SQL Server:

2.3: NoSQL (Ոչ ռելացիոն տվյալների բազաներ)

NoSQL-ը (սկզբնապես նշանակում է «ոչ միայն SQL» կամ «ոչ հարաբերական») վերաբերում է տվյալների բազայի դիզայնի այն տեսակին, որը պահպանում և վերականգնում է տվյալները ռելացիոն տվյալների բազաների ավանդական աղյուսակային կառուցվածքից տարբերվող ձևով: Ի տարբերություն ռելացիոն տվյալների բազաների, որոնք տվյալները կազմակերպում են տողերի և սյուների, ինչպես աղյուսակը, NoSQL տվյալների բազաները օգտագործում են մեկ տվյալների կառուցվածք՝ ինչպիսիք են բանալի-արժեք զույգերը, լայն սյուները, գրաֆիկները կամ փաստաթղթերը՝ տեղեկատվություն պահելու համար: Քանի որ այս ոչ հարաբերական դիզայնը չի պահանջում ֆիքսված սխեմա, այն հեշտությամբ մասշտաբավորվում է՝ կառավարելու մեծ, հաճախ ոչ կառուցվածքային տվյալների հավաքածուներ: NoSQL համակարգերը երբեմն անվանում են «Ոչ միայն SQL», քանի որ դրանք կարող են աջակցել SQL-անման հարցման լեզուներին կամ աշխատել SQL տվյալների բազաների հետ միասին բազմալեզու-համառ կարգավորումներում, որտեղ համակցված են տվյալների բազայի բազմաթիվ տեսակներ: Ոչ հարաբերական տվյալների բազաները սկիզբ են առնում 1960-ականների վերջից, բայց «NoSQL» տերմինը ի հայտ է եկել 2000-ականների սկզբին՝ խթանվելով Web 2.0 ընկերությունների, ինչպիսիք են սոցիալական մեդիա հարթակները, կարիքներով:

NoSQL տվյալների բազաները տարածված են մեծ տվյալների և իրական ժամանակի վեբ հավելվածներում՝ իրենց պարզ դիզայնի, մեքենաների կլաստերների միջև մասշտաբավորվելու ունակության (կոչվում է հորիզոնական մասշտաբավորում) և տվյալների մատչելիության ճշգրիտ վերահսկողության շնորհիվ: Այս կառուցվածքները կարող են արագացնել որոշակի առաջադրանքներ և հաճախ համարվում են ավելի հարմարվողական, քան ֆիքսված տվյալների բազայի աղյուսակները: Այնուամենայնիվ, շատ NoSQL համակարգեր առաջնահերթություն են տալիս արագությանը և մատչելիությանը խիստ համապատասխանության փոխարեն (CAP թեորեմի համաձայն), օգտագործելով վերջնական համապատասխանությունը, որտեղ թարմացումները հասնում են բոլոր հանգույցներին, սովորաբար միլիվայրկյանների ընթացքում, բայց կարող են առաջացնել կարճատև ուշացումներ վերջին տվյալներին մուտք գործելու հարցում, որոնք հայտնի են որպես հնացած ընթերցումներ: Չնայած մեծ մասը չունի ACID գործարքների լիարժեք աջակցություն, որոշները, ինչպես MongoDB-ն, այն ներառում են որպես հիմնական գործառնություն:

NoSQL բազաները նախագծված են ճկունության և բարձր մասշտաբայնության համար, ինչը հատկապես կարևոր է դինամիկ փոփոխվող տվյալների պարագայում:

- Ի տարբերություն SQL-ի, NoSQL բազաները չունեն ֆիքսված կառուցվածք: Տվյալները կարող են պահվել որպես փաստաթղթեր (JSON/BSON), բանալի-արժեք

գույգեր կամ գրաֆներ: Մա թույլ է տալիս նույն հավաքածուի (Collection) մեջ ունենալ տարբերվող դաշտերով օբյեկտներ:

- NoSQL համակարգերը հեշտությամբ մասշտաբավորվում են նոր սերվերներ ավելացնելու միջոցով (Sharding), ինչը թույլ է տալիս սպասարկել հսկայական քանակությամբ հարցումներ և տվյալներ:
- Քանի որ կարիք չկա կատարել աղյուսակների բարդ միավորումներ (Joins), տվյալների ընթերցումը և գրառումը կատարվում են չափազանց արագ:
- *Օրինակներ*՝ MongoDB, Cassandra, Redis, Amazon DynamoDB:

2.4: Ինչու՞ է այս նախագծի համար ընտրվել NoSQL (MongoDB)

Ավտոմեքենաների առքուվաճառքի հարթակի համար **MongoDB**-ի ընտրությունը պայմանավորված է մի քանի գործոններով.

1. Տարբեր տեսակի տրանսպորտային միջոցներ (օրինակ՝ էլեկտրական մեքենա, բեռնատար կամ մոտոցիկլետ) ունեն տարբեր տեխնիկական բնութագրեր: MongoDB-ի փաստաթղթային մոդելը թույլ է տալիս հեշտությամբ կառավարել այս տարբերությունները՝ առանց բազայի սխեմայի բարդ փոփոխությունների:
2. JavaScript-ի վրա հիմնված ստեկի (Node.js) հետ աշխատելիս MongoDB-ն ապահովում է բնական ինտեգրացիա, քանի որ տվյալները պահվում են BSON ձևաչափով, որը գրեթե նույնական է JSON-ին:
3. Հարթակի հետագա ընդլայնման և օգտատերերի քանակի աճի պարագայում MongoDB-ն թույլ է տալիս ապահովել բարձր հասանելիություն և արագագործություն:

Գլուխ 3: Վեբ-համակարգերի ճարտարապետական հիմունքները

Ժամանակակից վեբ-հավելվածների աշխատանքը հիմնված է հաճախորդ-սերվերային փոխազդեցության վրա, որտեղ հստակ տարանջատված են տվյալների ներկայացման (Frontend) և տվյալների մշակման ու պահպանման (Backend) գործառույթները:

3.1: Վեբ-բրաուզեր և Հաճախորդային մակարդակ

Վեբ-բրաուզերը (օրինակ՝ Google Chrome, Mozilla Firefox, Safari) հանդես է գալիս որպես օգտատիրոջ գործակալ, որի հիմնական գործառույթը վեբ-սերվերից ստացված տեղեկատվության կարգավորումն ու տեսանելի ցուցադրումն է: Երբ օգտատերը մուտքագրում է URL հասցեն, բրաուզերը կատարում է HTTP/HTTPS հարցում դեպի սերվեր և ստանում ֆայլերի հավաքածու, որոն ձևավորում են վեբ-կայքը:

Բրաուզերը ծրագրային ապահովում է, որը նախատեսված է վեբ-կայքեր դիտելու համար, այսինքն վեբ կայքերին դիմելու, դրանցից ստացված տեղեկատվությունը վերամշակելու, կայքը արտածելու և մի էջից մյուսն անցնելու համար:

Համացանցի ստեղծման օրվանից ցանցային զննիչները անընդհատ կատարելագործվել են և ժամանակի ընթացքում ավելի ու ավելի է մեծանում դրանց կարևորությունն ու պահանջարկը: Այժմյան ցանցային զննիչը վեբ-կայքի մշակման և արտածման համար նախատեսված ծրագրային համակարգ է, որն իրենից ներկայացնում է վեբ-կայքի և իր հաճախորդի միջև միջանկյալ օղակ, միջավայր: Գրեթե բոլոր հայտնի ցանցային զննիչները տարածվում են անվճար այլ լրացնող ծրագրերի հետ միասին. Microsoft Edge (Microsoft Windows-ի հետ միասին), Մոզիլլա Ֆայրֆոքս (անվճար, ազատ ծրագրային ապահովում), Սաֆարի (Mac OS-ի հետ կամ անվճար Windows-ի համար), Օպերա (անվճար սկսած 8.50 տարբերակից), Գուգլ Բրոմ (անվճար, ազատ ծրագրային ապահովում):

2019 թվականի մարտի տվյալներով ավելի քան 4.3 մլն մարդ օգտագործում է զննիչ, որը մոտավորապես կազմում է աշխարհի բնակչության 55%-ը: Ամենատարածված զննիչներն են Գուգլ Բրոմ-ը, Մոզիլլա Ֆայրֆոքս-ը, Սաֆարի-ն և Microsoft Edge-ը:

Համացանցային զննիչի նպատակը Համաշխարհային սարդոստայն-ից տեղեկատվությունը հասցնել և ցույց տալ դա Օգտվող-ի սարքի վրա:

Այս գործընթացը սկսում է երբ օգտագործողը ներմուծում է URL-ը զննիչում, օրինակ <https://en.wikipedia.org/>: Գործնականում բոլոր URL-ները սկսում են http:-ով կամ https:-ով, ինչը նշանակում է, որ զննիչը կստանա դրանք որպես HTTP: https:-ի դեպքում զննիչի և վեբ

սպասարկիչի միջև հաղորդակցումը կողավորված է անվտանգության և գաղտնիության սկզբունքներից ելնելով: URL-ի համար մեկ այլ նախամասնիկ է file:-ը, որը օգտագործվում է համակարգչի ներքին պանակներն ու նիշքերը ցույց տալու համար:

Մի անգամ կայքէջը բեռնվում է դիատրկիչում, և զննիչը ցույց է տալիս այն օգտագործողի սարքի էկրանին: Այն ներառում է զննիչի կողմից ներառվող հնարավոր պատկերները և վիդեոները:

Վեբ էջը Համաշխարհային սարդոստայնի մաս է, որը նախատեսված է վեբ զննարկիչի միջոցով այցելուներին հասանելի լինելու համար: Այն ունի վեբ հասցե: Տեխնիկապես, վեբ էջը հաճախ կազմված է HTML (Hypertext Markup Language) կամ XHTML փաստաթղթից և նկարներից: Սակայն վեբը կարող է պարունակել ցանկացած տեսակի աղբյուրներ՝ տեքստային, տեսային, ձայնային, ծրագրային:

Վեբ-էջի հիմնական բաղադրիչներն են.

3.2: HTML (HyperText Markup Language)

Կազմում է էջի կառուցվածքը և բովանդակությունը (վերնագրեր, պարբերություններ, ձևաչափեր):

HTML-ի օգնությամբ հեշտությամբ կարելի է ստեղծել համեմատաբար պարզ, սակայն գեղեցիկ ձևավորված փաստաթուղթ: Ի հավելումն փաստաթղթի կառուցվածքի պարզեցմանը՝ HTML-ում աջակցվում էր հիպերտեքստը: Մուլտիմեդիայի հնարավորությունները ավելացվեցին քիչ ավելի ուշ: Ի սկզբանե HTML լեզուն ստեղծվել է որպես փաստաթղթերի կառուցման և ձևավորման միջոց՝ անկախ օգտագործվող արտածման միջոցից (էկրան, տպիչ և այլն) ցուցադրելու միջոց: Նախնական HTML ձևաչափով տեքստը պետք է առանց ռճային և կառուծվածքային ձևափոխման արտածվեր սարքավորման վրա՝ տարբեր տեխնիկական համալրմամբ (ժամանակակից համակարգչի գունավոր էկրանը, օրգանայզերի մոնիթրոմային էկրան, բջջային հեռախոսների կամ տեքստերի ձայնային վերարտադրման ծրագրերի կամ սարքավորումների սահմանափակ չափերով էկրան): Սակայն HTML-ի ժամանակակից կիրառումը հեռու է սկզբնական նշանակությունից:

Տեքստային փաստաթղթերը, որոնք պարունակում են HTML կոդ (այդ փաստաթղթերը հիմնականում ունեն .html կամ .htm ընդլայնում), մշակվում են հատուկ ծրագրերի միջոցով, որոնք ցուցադրում են փաստաթուղթը իր ձևավորման համաձայն: Այդ ծրագրերը, որոնք անվանում են դիտարկիչներ (browser), սովորաբար իրենցից ներկայացնում են հարմար միջավայր վեբ էջերի հայցման, դրանց դիտման և անհրաժեշտության դեպքում նաև դեպի սերվեր տվյալների փոխանցման համար: Այժմ

աշխարհում առավել գործածվող դիտարկիչներն են՝ Մոզիլա Ֆայրֆոքս, Գուգլ Բրոմ, Ինտերնետ Էքսպլորեր, Օպերա և Սաֆարի:

HTML լեզվով գրված ցանկացած փաստաթուղթ իրենից ներկայացնում է իրար մեջ ներդրված տարրերի հավաքածու (*HTML element*), ընդ որում բոլոր տարրերի սկիզբը և վերջը նշվում է հատուկ նշիչներով՝ թեգերով (*tag*): Տարրերը կարող են լինել դատարկ, այսինքն՝ չպարունակեն ոչ մի տեքստ կամ տվյալ: Օրինակ `<br />` թեգը ջարդում է տողը՝ հաջորդող տեքստը սկսելով նոր տողից (բայց ոչ նոր պարբերությունից): Այս դեպքում սովորաբար չի նշվում փակող թեգ: Բացի այդ, տարրերը կարող են պարունակել հատկանիշներ (HTML ատրիբուտներ), որոնք որոշում են նրանց որոշակի հատկանիշներ, օրինակ տառաչափը *font* տարրի համար: Ատրիբուտները նշվում են բացող թեգի մեջ: HTML փաստաթուղթը սկսվում է `<html>` թեգով և ավարտվում է `</html>` թեգով: Փաստաթուղթը բաղկացած է երկու հիմնական մասերից՝ ծանուցման բաժին և փաստաթղթի մարմին: Ծանուցման բաժինը ներառվում է `<head></head>` թեգերի մեջ: Այս բաժնում ներառվում են նաև տեխնիկական այլ բնութագրեր: Էջի վերնագիրը տրվում է `<title> ..</title>` մեկնարկի և ավարտի թեգերի միջև: Html-փաստաթղթի մարմինը ներառվում է `<body>` և `</body>` զույգ թեգերի մեջ: Գրանցումները, որտեղ նշվել են տարրի անվանումը կամ ատրիբուտները, կարևոր չեն (բացի XHTML -ից): Տարրերը կարող են լինել ներդրված:

HTML փաստաթղթեր ստեղծելու համար օգտագործվում են HTML մասնիկներ, որոնք իրենցից ներկայացնում են HTML-ի թեգեր պարփակված ուղղանկյուն փակագծերի մեջ: Օրինակ՝ `<p>`:

Դեպքերի մեծ մասում մասնիկները հանդիպում են զույգ թեգերով, որոնք են "բացման թեգը" `<p>` և "փակման թեգը" `</p>`, որի մեջ դրվում է `" / "` նշանը: Ներկայացվող ինֆորմացիան գրվում է այդ երկու թեգերի արանքում:

Որոշ դեպքերում բացման թեգերը իրենց մեջ պարունակում են լրացուցիչ ատրիբուտներ, որոնք օգտագործվում են հավելյալ ինֆորմացիա ներկայացնելու համար: Հիմնական դեպքերում դրանք օգտագործվում են HTML փաստաթղթին այլ ֆորմատի նյութեր կցելու համար նախատեսված թեգերի հետ կամ մասնիկի մեջ ներկայացված ինֆորմացիայի կոորդինատները նշելու համար: Օրինակ՝ `<img>`:

Առանձին մասնիկներ, ինչպիսին է տողընդհատը `<br>`, իրենց մեջ չեն պարունակում տեքստ կամ այլ թեգեր և պարունակում են միայն մեկ դատարկ բացման թեգ՝ առանց փակման թեգի:

Մասնիկների մի մասը կամավոր օգտագործման են և նրանց օգտագործումը փաստաթղթի մեջ ոչինչ չի փոխում: Այդպիսին է օրինակ պարբերության մասնիկը `<p>`: Այդ մասնիկները օգտագործվում են հին բրաուզերների համար ստեղծվող կայքերի մեջ:

Այնուամենայնիվ, HTML մասնիկի հիմնական տեսքն է <թեգ ատրիբուտ1="արժեք1" ատրիբուտ2="արժեք2">պարունակություն</թեգ>-ը: Իսկ դատարկ մասնիկների դեպքում, ինչպիսին են
 և մասնիկները <թեգ ատրիբուտ1="արժեք1" ատրիբուտ2="արժեք2">-ը: Դատարկ մասնիկների մեջ փակման թեգի օգտագործումը ո՛չ անհրաժեշտ է, ո՛չ էլ թույլատրելի: Իսկ, եթե ատրիբուտին արժեք տրված չէ, օգտագործվում են հիմնական արժեքները:

3.3: CSS (Cascading Style Sheets)

Պատասխանատու է տեսողական ձևավորման, գունային լուծումների և էջի տարրերի դասավորվածության համար՝ ապահովելով էկրանի տարբեր չափսերին հարմարվողականությունը:

CSS, կամ կասկադային ոճաթերթ (Cascading Style Sheets - կասկադային ոճերի թերթներ), կայքին ձևավորում տալու համար օգտագործվող տեխնոլոգիա ծրագրավորման ֆորմալ լեզվով: CSS-ը հիմնականում օգտագործվում է HTML և XHTML ֆորմատի պատկանող վեբ-էջերի ձևավորման համար, սակայն կարող է նաև օգտագործվել ցանկացած տեսակի XML ֆորմատի փաստաթղթերի համար, ներառյալ SVG և XUL, ինչպես նաև դիզայնի, մակետի, տարբեր սարքերի և տարբեր էկրանների վրա ներկայացնելու համար:

Երբ HTML 3.2-ի սպեցիֆիկացիայի մեջ ավելացվեցին -ի նման թեգերը, գույնի ատրիբուտները: Խոշոր վեբ-կայքերի նախագծումը դարձավ երկար և ծախսատար գործընթաց, քանի որ ամեն էջի վրա պետք է նշվեր շրիֆտը և գունային ինֆորմացիան: Այս խնդիրը լուծելու համար World Wide Web (W3C) կոնսորցիումը ստեղծեց CSS-ը: CSS-ը HTML-ի էջից հանեց ոճային ֆորմատավորումը:

CSS-ը օգտագործվում է վեբ-էջեր նախագծողների կողմից գույներ, տառատեսակներ, գտնվելու վայր և այլ նկարագրություններ տալու համար: CSS-ի հիմնական նպատակն է առանձնացնել HTML էջի պարունակությունը և նրա նկարագրությունը, դրանով իսկ էջին հաղորդել մատչելիություն, ճկունություն և նրա նկարագրությունը կառավարելու հնարավորություն, ինչպես նաև նվազեցնել հրամանների կրկնությունը և բարդությունը: Բացի այդ CSS-ը թույլ է տալիս տվյալ դրկումենտը ներկայացնել տարբեր ոճերով: Էջի արտածման ժամանակ CSS-ը կարող է վերցվել տարբեր աղբյուրներից:

CSS Դասերը և նույնացուցիչները կարող են վերաբերվել միայն նշված տարրերին (թեգերին), որոնք HTML կոդում ունեն ատրիբուտներ՝ class կամ id: Հիմնական տարբերությունն այն է, որ նույնացուցիչը (id) նշանակվում կամ սահմանվում է մեկ էլեմենտի համար, իսկ դասը (class)-ը սովորաբար հատկանշում է մի քանի էլեմենտներ: Այնուամենայնիվ, ժամանակակից բրաուզերները կարող են տարբերակել մի քանի էլեմենտներ միևնույն իդենտիֆիկատորով: Տարբերությունն այն է նաև, որ կարող են լինել մի քանի դասեր, որոնք ծրագրի կոդում առանձնացվում են բացատանիշներով, իսկ

նույնացուցիչները այդպես ներկայացնել հնարավոր չէ: Կարևոր է նշել հետևյալ տարբերությունը՝ նույնացուցիչները լայնորեն կիրառվում են JavaScript ծրագրի կոդում, որպեսզի գտնեն փաստաթղթի ունիկալ տարրերը: Ի տարբերություն թեգերի և նրանց ատրիբուտների, դասերի և նույնացուցիչների անունները կախված են կոդում տառերի գրառման ձևից: Դասերի և նույնացուցիչների հատկությունները սահմանվում են սելեքթորների օգնությամբ: Կարող է դասի հատկությունը տրվել ամբողջությամբ (այդ դեպքում սելեքթորը սկսվում է «.» նշանով), կամ նույնացուցիչն ինքն իր հատկությամբ (այս դեպքում սելեքթորը սկսվում է «#» նշանով), ինչպես նաև այդ դասի ցանկացած էլեմենտի կամ նույնացուցիչի հատկությունով:

3.4: JavaScript

Ապահովում է էջի դինամիկան և ինտերակտիվությունը: Այս նախագծում օգտագործված React գրադարանը թույլ է տալիս կառուցել բարդ ինտերֆեյսներ՝ հիմնված բազմակի օգտագործման բաղադրիչների վրա: React-ի հիմնական գաղափարն այն է, որ ինտերֆեյսը բաժանվում է փոքր, ինքնուրույն կառավարվող հատվածների, ինչը զգալիորեն հեշտացնում է ինչպես կոդի կազմակերպումը, այնպես էլ հետագա սպասարկումն ու ընդլայնումը: Բացի այդ, React-ը օգտագործում է վիրտուալ DOM (Virtual DOM)՝ մեխանիզմ, որի շնորհիվ էջի թարմացումները կատարվում են առավել արդյունավետ՝ թարմացնելով միայն անհրաժեշտ հատվածները, ոչ թե ամբողջ էջը:

ՋավաՍկրիպտը ինտերպրետացվող ծրագրավորման լեզու է, որը հանդիսանում է ժամանակակից վեբ մշակման հիմնասյուներից մեկը: Այն համացանցում ամենահանրահայտ սկրիպտային լեզուն է և աշխատում է բոլոր հանրահայտ բրաուզերներում, որոնց թվում են Google Chrome, Mozilla Firefox, Opera և Safari: JavaScript-ի օգնությամբ հնարավոր է իրականացնել դինամիկ փոփոխություններ էջում՝ առանց այն ամբողջությամբ վերաբեռնելու, ինչպես նաև կազմակերպել օգտատիրոջ գործողությունների մշակում (օրինակ՝ սեղմումներ, մուտքագրումներ, ձևերի վավերացում և այլն):

React գրադարանը հանդիսանում է JavaScript էկոհամակարգի կարևոր բաղադրիչներից մեկը և լայնորեն օգտագործվում է Single Page Application (SPA) տիպի հավելվածների ստեղծման համար: Այն ունի հարուստ էկոհամակարգ՝ ներառյալ տարբեր լրացուցիչ գրադարաններ և գործիքներ, ինչպիսիք են վիճակի կառավարման համար նախատեսված լուծումները (օրինակ՝ Redux կամ Context API), ինչպես նաև երթուղավորման (routing) գրադարանները, որոնք թույլ են տալիս կազմակերպել էջերի միջև անցումները առանց ամբողջական վերաբեռնման: React-ը նաև խրախուսում է դեկլարատիվ ծրագրավորում, որտեղ մշակողը սահմանում է, թե ինչ պետք է արտացոլվի ինտերֆեյսում տվյալ վիճակում, իսկ գրադարանը ինքն է հոգում դրա արդյունավետ ցուցադրման մասին:

Այս նախագծում կիրառվել է նաև TypeScript լեզուն, որը հանդիսանում է JavaScript-ի ընդլայնված տարբերակ՝ ավելացնելով ստատիկ տիպավորում (static typing): TypeScript-ի հիմնական առավելությունն այն է, որ այն թույլ է տալիս նախապես հայտնաբերել բազմաթիվ սխալներ՝ դեռևս ծրագրի մշակման փուլում, ինչի շնորհիվ բարձրանում է կոդի հուսալիությունը և նվազում են runtime-ի ընթացքում առաջացող խնդիրները: Բացի այդ, TypeScript-ը հեշտացնում է մեծածավալ նախագծերի կառավարումը, քանի որ հստակ տիպերի սահմանումը դարձնում է կոդը ավելի ընթեռնելի և կանխատեսելի թիմային աշխատանքի ընթացքում:

React-ի և TypeScript-ի համադրությունը հանդիսանում է ժամանակակից վեբ մշակման արդյունավետ լուծում, քանի որ այն միավորում է ճկունությունը և արագագործությունը՝ միաժամանակ ապահովելով կոդի որակն ու կայունությունը: Այս մոտեցումը հատկապես օգտակար է բարդ և մեծածավալ համակարգերի դեպքում, որտեղ կարևոր է ինչպես կատարողականությունը, այնպես էլ երկարաժամկետ սպասարկելիությունը:

Գլուխ 4: Վեբ-սերվեր և Սերվերային մակարդակ

4.1: Սերվեր

Սերվերը (server), մասնագիտացված սարքերի համակարգ է, որը թույլ է տալիս աշխատել ոչ միայն լոկալ, այլև առցանց: Սերվերը հենց նույն համակարգիչն է, սակայն սովորաբար այն ավելի հզոր է լինում:

Սերվերը ինչպես և համակարգիչը բաղկացած է մայր պլատայից, պրոցեսորից, օպերատիվ հիշողությունից, կոշտ սկավառակից: Հիմնականում սերվերներում օգտագործվում են հատուկ սարքեր, բայց կան նաև այնպիսի սերվերներ որոնք ունեն սովորական համարկզյի մասեր:

Բացի օպերացիոն համակարգի տեղադրումից մնացած այլ գործողությունները սերվերի վրա հնարավոր է կատարել առցանց, այլ ոչ թե տեղում: Սերվերների հիմնական և ամենակարևոր դերն այն է, որ սերվերները ապահովում են ինտերնետ պրովայդերների աշխատանքի կազմակերպումը: Սերվերը միացնում է օգտագործողին և նրան է փոխանցում ինտերնետ թրաֆիկը, այդպիսով հանդիսանալով յուրօրինակ բազա: Բացի այդ գոյություն ունեն DNS սերվերներ, որոնք պատասխանատու են սիմվոլային URL-ի ձևափոխմանը IP հասցեների: Պրովայդերի ցանկացած ծառայություն աշխատում է հիմնվելով սերվերի վրա:

Սերվերների ամենատարածված նշանակություններից է տվյալների պահպանումը, օրինակ սերվերի վրա է պահվում փոստային համակարգից օգտվողների բոլոր նամակները: Ինչպես նաև կարող են պահվել օգտվողների տարբեր տվյալներ կախված ծառայությունից: Երրորդ և ամենատարածված նշանակությունը դա հոստինգն է: Հոստինգը դա սերվեր է որի վրա պահվում են կայքերը և այն բոլոր ֆայլերը որոնք տեղադրված են այդ կայքում:

Սերվերը հզոր հաշվողական համակարգ է կամ ծրագրային միջավայր, որը մշտապես գտնվում է ակտիվ վիճակում՝ սպասելով հաճախորդներից ստացվող հարցումներին: Սերվերային շերտի հիմնական խնդիրն է իրականացնել բիզնես-տրամաբանությունը և ապահովել տվյալների անվտանգությունը:

4.2: Node.js

Այս նախագծում սերվերային հատվածի իրականացման համար կիրառվել է Node.js միջավայրը: Node.js-ը հանդիսանում է JavaScript-ի գործարկման միջավայր, որը թույլ է տալիս JavaScript լեզուն օգտագործել ոչ միայն բրաուզերում, այլ նաև սերվերի կողմում: Այն կառուցված է Google Chrome-ի V8 շարժիչի վրա, ինչի շնորհիվ ապահովում է բարձր կատարողականություն և արագ տվյալների մշակում:

Node.js-ի հիմնական առանձնահատկություններից մեկը ոչ բլոկավորող (non-blocking) և իրադարձահեն (event-driven) ճարտարապետությունն է: Սա նշանակում է, որ սերվերը կարող է միաժամանակ մշակել բազմաթիվ հարցումներ՝ առանց սպասելու յուրաքանչյուր գործողության ավարտին, ինչը հատկապես կարևոր է մեծ ծանրաբեռնվածությամբ համակարգերի համար: Այդ մոտեցումը զգալիորեն բարձրացնում է համակարգի արդյունավետությունը և թույլ է տալիս սպասարկել մեծ թվով օգտվողների միաժամանակ:

Բացի այդ, Node.js-ն ունի հարուստ էկոհամակարգ՝ ներառյալ բազմաթիվ գրադարաններ և փաթեթներ, որոնք հասանելի են npm (Node Package Manager)-ի միջոցով: Սա արագացնում է մշակման գործընթացը և հնարավորություն է տալիս օգտագործել արդեն պատրաստի լուծումներ տարբեր խնդիրների համար, ինչպիսիք են սերվերի երթուղավորումը, տվյալների մշակումը և անվտանգությունը:

Node.js-ը լայնորեն կիրառվում է RESTful API-ների, իրական ժամանակում աշխատող հավելվածների (օրինակ՝ չաթեր կամ ծանուցումների համակարգեր) և միկրոսերվիսային ճարտարապետությամբ համակարգերի կառուցման համար՝ ապահովելով ճկուն, մասշտաբավորվող և ժամանակակից սերվերային լուծում:

Այս նախագծի շրջանակներում սերվերն իրականացնում է հետևյալ գործառնությունները.

- Ընդունում է API հարցումները բրաուզերից, վավերացնում է դրանք և որոշում հետագա գործողությունների ընթացքը:
- Անհրաժեշտության դեպքում սերվերը կապ է հաստատում MongoDB-ի հետ՝ տվյալներ կարդալու (օրինակ՝ մեքենաների ցուցակը ստանալու) կամ գրելու (նոր հայտարարություն ավելացնելու) համար:
- Ստուգում է օգտատիրոջ ինքնությունը և իրավունքների մակարդակը՝ նախքան զգայուն տվյալներին հասանելիություն տրամադրելը:
- Մշակված տվյալները վերադարձնում է բրաուզերին՝ հիմնականում JSON ձևաչափով, որպեսզի Frontend-ը կարողանա դրանք արտապատկերել էկրանին:

Գլուխ 5: Կառուցվածքը և Ինչպես օգտվել կայքից

5.1 Բազայի կառուցվածքը



Այս պրոեկտի տվյալների բազան կազմված է 3 փոխկապակցված հավաքածուներից՝

- users: Օգտատերերի տվյալների հավաքածուն:
- cars: Գնորդների զցած ավտոմեքենաների հավաքածուն:
- carapplications: Հարցումների հավաքածուն, որ տեղի են ունեցել գնորդների և վաճառողների միջև:

5.1.1: Օգտատերեր

Օգտատերերը կարող են լինել 2 տիպի՝ գնորդ և վաճառող: Օգտատերերին տարբեր հասանելիություններ տրամադրելու համար ներմուծված է role դաշտը, որտեղ պահվում են 2 արժեքներ՝ buyer և seller, ըստ որոնց էլ տրամադրվում է հասանելիությունը:

```
_id: ObjectId('69b732e295d21ae32de6c638')
name: "Edgar Harutyunyan"
email: "duotech.biz@gmail.com"
image: "https://res.cloudinary.com/dnebj68wn/image/upload/v1773613793/ijl6v2ww..."
role: "buyer"
password: "$2b$10$bS2kup3QzU/B8r0iweJ4.B6kD.ER3aGn4WpVaqJlMWV//TnBzWK"
__v: 0
```

```
_id: ObjectId('69b882c2897a7fb9ac57cc4f')
name: "Grigor O"
email: "ohanyang183@gmail.com"
image: "https://res.cloudinary.com/dnebj68wn/image/upload/v1773699776/wzzolj3n..."
role: "seller"
password: "$2b$10$.PoUtKk7YSi97T9v9jRI8uCe5Kg1BB1SHayo7U8tqfQXGZWuIg9Mu"
__v: 0
```

Առաջին հայացքից կարող է թվալ, որ համակարգը կարելի է պարզեցնել՝ օգտագործելով `boolean` տիպով դաշտ (օրինակ՝ `seller / not seller`), սակայն նման մոտեցումը երկարաժամկետում սահմանափակումներ է ստեղծում: Հետագայում հնարավոր է անհրաժեշտություն առաջանա ավելացնել նոր դերեր՝ օրինակ `admin`, `moderator` կամ այլ հատուկ օգտատերերի խմբեր, և այդ դեպքում համակարգը արդեն չի կարող սահմանափակվել միայն երկու արժեքով:

Ակնհայտ է նաև, որ օգտատիրոջ գաղտնաբառը չի պահպանվում իր սկզբնական տեսքով: Այն անցնում է գաղտնագրման (`hashing`) գործընթացով, ինչը ապահովում է տվյալների անվտանգությունը: Այս դեպքում գաղտնաբառը մշակվում է երկու փուլով՝ նախ կիրառվում է աղ (`salt`), ապա ստացված արժեքը ենթարկվում է հեշավորման ալգորիթմի, ինչի արդյունքում ստացվում է եզակի և չվերականգնվող արժեք, որը պահվում է տվյալների բազայում:

```
const salt = await bcrypt.genSalt(10);  
const hashPassword = await bcrypt.hash(password, salt);
```

Salting. այս փուլում գեներացվում է պատահական սիմվոլներից բաղկացած տող (`salt`), որը ավելացվում է գաղտնաբառին՝ մինչև հեշավորման գործընթացը սկսելը: Սա ապահովում է, որ նույնիսկ նույն գաղտնաբառը տարբեր օգտատերերի համար կունենա տարբեր արդյունք՝ բարձրացնելով անվտանգության մակարդակը:

Hashing. այս փուլում արդեն ստացված (գաղտնաբառ + `salt`) տողը փոխանցվում է միակողմանի ֆունկցիայի միջոցով: Այդ ֆունկցիան չի կարող շրջադարձվել, այսինքն՝ հնարավոր չէ ստանալ սկզբնական գաղտնաբառը հեշից: Արդյունքում ստացվում է վերջնական հեշ արժեքը, որը պահպանվում է տվյալների բազայում և օգտագործվում է մուտքի ժամանակ ստուգման համար:

5.1.2: Ավտոմեքենաներ

Ավտոմեքենաների հավաքածույի մեջ գտնվող օբյեկտներն ունեն բազմաթիվ դաշտեր, որոնցից առանձնահատուկ է `sellerID` դաշտը: Յուրաքանչյուր ավտոմեքենայի հայտարարություն ստեղծվում է ինչ որ վաճառողի կողմից, և այդ տեղեկությունը պետք է պահպանվի հայտարարությանը համապատասխան՝ բազայում գտնվող օբյեկտի ինչ դաշտերից մեկում:

```

_id: ObjectId('69bbdeef118504321983739')
title: "2020 BMW X5 xDrive40i"
description: "<p>Luxury mid-size SUV with full service history, excellent condition,..."
location: "Yerevan"
category: "SUV"
condition: "Good"
price: 62000
image: "https://res.cloudinary.com/dnebj68wn/image/upload/v1773919980/cars/z2n..."
date: 1773919982203
visible: true
sellerId: ObjectId('69b882c2897a7fb9ac57cc4f')
> specs: Object
__v: 0

```

Սա մեզ կօգնի ճիշտ կազմակերպել տվյալների պահպանումը համակարգում, քանի որ յուրաքանչյուր օբյեկտ կունենա հստակ սահմանված կառուցվածք և կպահպանի միայն իրեն վերաբերող տեղեկությունները: Օրինակ՝ ավտոմեքենայի հայտարարության դեպքում sellerID դաշտը թույլ կտա անմիջապես հասկանալ, թե որ վաճառողին է պատկանում տվյալ հայտարարությունը, առանց լրացուցիչ որոնումների կամ բարդ կապերի:

Դիտարկենք մի իրավիճակ, որտեղ համակարգում կան հարյուրավոր հայտարարություններ տարբեր վաճառողներից: Այս մոտեցումը թույլ կտա արագ գտնել տվյալները ըստ վաճառողի, ցուցադրել կոնկրետ օգտատիրոջ բոլոր հայտարարությունները և հեշտացնել տվյալների կառավարումը: Արդյունքում համակարգը դառնում է ավելի կազմակերպված, արդյունավետ և հեշտ ընդլայնվող:

5.1.3: Գործարքներ (դիմումներ)

Դիմումների հավաքածույի մեջ յուրաքանչյուր ունի մի քանի դաշտեր, որոնցից առանձնահատուկ են երեք օտար բանալիները՝ userId, sellerId և carId: Այս դաշտերը օգտագործվում են տարբեր հավաքածուների միջև կապ ստեղծելու համար: userId դաշտը ցույց է տալիս հայտը ստեղծած օգտատիրոջը, sellerId դաշտը՝ այն վաճառողին, ում ավտոմեքենային վերաբերում է տվյալ հայտը, իսկ carId դաշտը՝ կոնկրետ ավտոմեքենային:

```
_id: ObjectId('69bbe48a10e6cebb97bcf856')
userId: ObjectId('69b732e295d21ae32de6c638')
sellerId: ObjectId('69b882c2897a7fb9ac57cc4f')
carId: ObjectId('69bbdeeeef118504321983739')
status: "Rejected"
date: 1773921418936
__v: 0
```

Այս կառուցվածքը թույլ է տալիս համակարգին չկրկնել տվյալները մեկ օբյեկտի ներսում և փոխարենը պահել միայն հղումներ դեպի համապատասխան հավաքածուները: Օրինակ՝ մեկ օգտատեր կարող է ստեղծել մի քանի հայտ տարբեր մեքենաների համար, իսկ նույն մեքենան կարող է կապ ունենալ տարբեր օգտատերերի հայտերի հետ: Այդ պատճառով այս երեք դաշտերը ապահովում են տվյալների միջև ճիշտ կապը և համակարգի ավելի կազմակերպված աշխատանքը:

Ամփոփելով՝ համակարգի տվյալների կառուցվածքը և անվտանգության մոտեցումները նախագծված են այնպես, որ ապահովեն և՛ ճկունություն, և՛ մասշտաբելիություն:

Օգտատերերի դեպքում կիրառվում է role դաշտի վրա հիմնված տարբերակում, ինչը թույլ է տալիս հեշտ կառավարել հասանելիությունները և ապագայում առանց կառուցվածքային փոփոխությունների ավելացնել նոր դերեր: Միաժամանակ գաղտնաբառերի մշակման համար կիրառվում է salting + hashing մեխանիզմը, որը ապահովում է բարձր մակարդակի անվտանգություն՝ բացառելով գաղտնաբառերի վերականգնումը կամ պարզ ընթերցումը տվյալների բազայից:

Ավտոմեքենաների հավաքածուում յուրաքանչյուր հայտարարություն կապվում է իր ստեղծողի հետ՝ sellerID դաշտի միջոցով, ինչը թույլ է տալիս արդյունավետ կերպով կազմակերպել տվյալները, արագ գտնել կոնկրետ վաճառողի հայտարարությունները և նվազեցնել ավելորդ կրկնությունները համակարգում:

Գործարքների (դիմումների) հատվածում կիրառվում է օտար բանալիների համակարգ՝ userId, sellerId և carId, որոնք ապահովում են տարբեր հավաքածուների միջև հստակ կապ: Այս մոտեցումը թույլ է տալիս մեկտեղել տարբեր տվյալներ առանց կրկնօրինակման, ինչպես նաև հեշտացնում է տվյալների որոնումը և կառավարումը:

Ընդհանուր առմամբ, այս կառուցվածքը դարձնում է համակարգը ավելի կազմակերպված, անվտանգ և հեշտ ընդլայնվող՝ ապահովելով ինչպես ներկայիս, այնպես էլ ապագա պահանջների աջակցում:

5.2 Ինչպես օգտվել կայքից

 AutoMarket

Seller Login [Login](#)

Over 10,000+ cars sold

Find your next ride or sell your car fast
Explore the best deals and start your journey!

[Search](#)

We Sell



Search by Categories

Search by Location

Search by Specs

Fuel type

Any

Drivetrain

Any

Steering

Any

Condition

+ Latest cars

Get your desired car



2019 Honda Civic Type R

Yerevan Hatchback

Price: \$43000

[Buy Now](#) [See details](#)



2019 Subaru Impreza

Yerevan Sedan

Price: \$19000

[Buy Now](#) [See details](#)



2012 Lexus RX 350

Vanadzor SUV

Price: \$13000

[Buy Now](#) [See details](#)

Կայք մուտք գործելուն պես կցուցադրվի հիմնական էջը, որտեղ ներկայացված են բոլոր տեղադրված հայտարարությունները՝ հարմար և հասկանալի ձևաչափով: Օգտատերը կարող է հեշտությամբ դիտել առկա ավտոմեքենաները, ծանոթանալ դրանց հիմնական տվյալներին և ընտրել իրեն հետաքրքրող տարբերակները: Բացի այդ, կայքում առկա է որոնման համակարգ, որի միջոցով հնարավոր է արագ գտնել ցանկալի ավտոմեքենան՝ մուտքագրելով համապատասխան բանալի բառեր: Որոնման գործընթացը ավելի արդյունավետ դարձնելու համար հասանելի են նաև տարբեր ֆիլտրեր, որոնք թույլ են տալիս սահմանափակել արդյունքները՝ ըստ գնի, արտադրության տարեթվի, մակնիշի, մոդելի և այլ կարևոր չափանիշների:

21

Over 10,000+ cars sold

Find your next ride or sell your car fast
Explore the best deals and start your journey!

Subaru

Yerevan

Search

We Sell



Current Search

Subaru X

Search by Categories

Search by Location

☒ Yerevan

☐ Gyumri

☐ Vanadzor

☐ Kapan

☐ Exeghnadzor

☐ Artashat

☐ Abovyan

Latest cars

Get your desired car



2019 Subaru Impreza

Yerevan Sedan

Price: \$19000

Buy Now

See details

Հայտարարությունների պաստառները պարունակում են ավտոմեքենայի մասին հիմնական, կարճ տեղեկատվություն՝ ներառյալ նկարը, անվանումը, տիպը, գտնվելու վայրը և գինը, ինչը թույլ է տալիս արագ կողմնորոշվել առաջարկների մեջ և համեմատել տարբեր տարբերակներ:

Ցանկալի ավտոմեքենան գտնելուց հետո օգտատերը կարող է սեղմել «Գնել» կամ «Մանրամասներ» հղման վրա, ինչի արդյունքում կտեղափոխվի տվյալ ավտոմեքենայի առանձին էջ: Այդ էջում ներկայացված է ավելի ամբողջական տեղեկատվություն՝ ավտոմեքենայի տեխնիկական և լրացուցիչ պարամետրերի մասին, ինչպես նաև տրամադրվում է հնարավորություն շարունակելու և ավարտին հասցնելու գնման գործընթացը:



2019 Subaru Impreza



Grigor O



Yerevan



Good



Price: 19k

Buy the car

Posted a month ago

Car Description

Compact AWD sedan, great for winter conditions.

Specifications

Brand	Subaru	Model	Impreza
Year	2019	Mileage	50,000 km
Transmission	Automatic	Fuel type	Petrol
Color	Silver	Seats	5
Engine size	2 L	Horsepower	152 HP
Torque	197 Nm	Cylinders	4
Fuel consumption	7.1 L/100km	Gears	8
Drivetrain	AWD	Steering	Left

More from Grigor O



2020 BMW X5 xDrive40i

Yerevan

SUV

Price: \$62000

Buy Now

See details



Տեղափոխվելով տվյալ ավտոմեքենայի էջ՝ կցուցադրվի այդ ավտոմեքենայի վաճառողի մասին տեղեկատվությունը, ավտոմեքենայի տեխնիկական պարամետրերը, վիճակը, ինչպես նաև նույն վաճառողի կողմից տեղադրված այլ հայտարարությունները, ինչը հնարավորություն է տալիս ավելի լավ պատկերացում կազմել ինչպես մեքենայի, այնպես էլ վաճառողի մասին:

Եթե ավտոմեքենան բավարարում է ձեր պահանջներին և սեղմեք «Գնել» կոճակը, ապա կցուցադրվի զգուշացում, որը կհուշի, որ անհրաժեշտ է մուտք գործել համակարգ կամ բացել ձեր գնորդի անձնական էջը՝ գնման գործընթացը շարունակելու համար:

Seller Login

Login

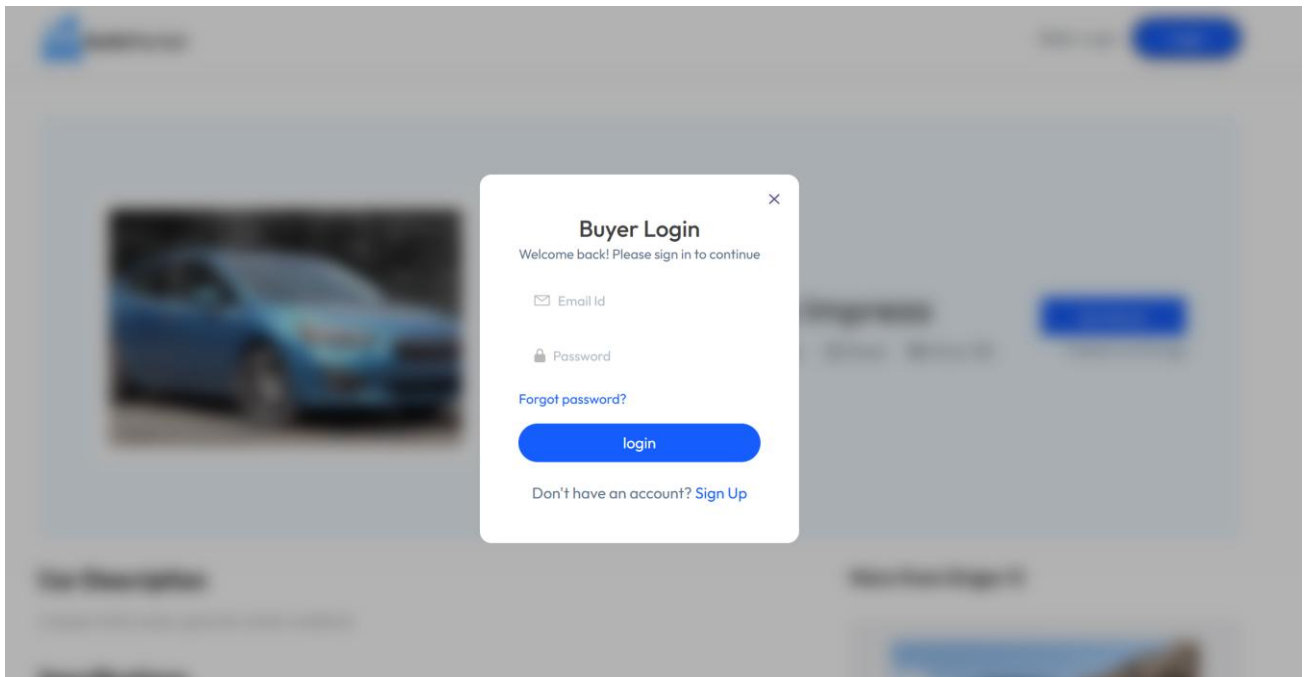


Login as a buyer to express interest
in cars

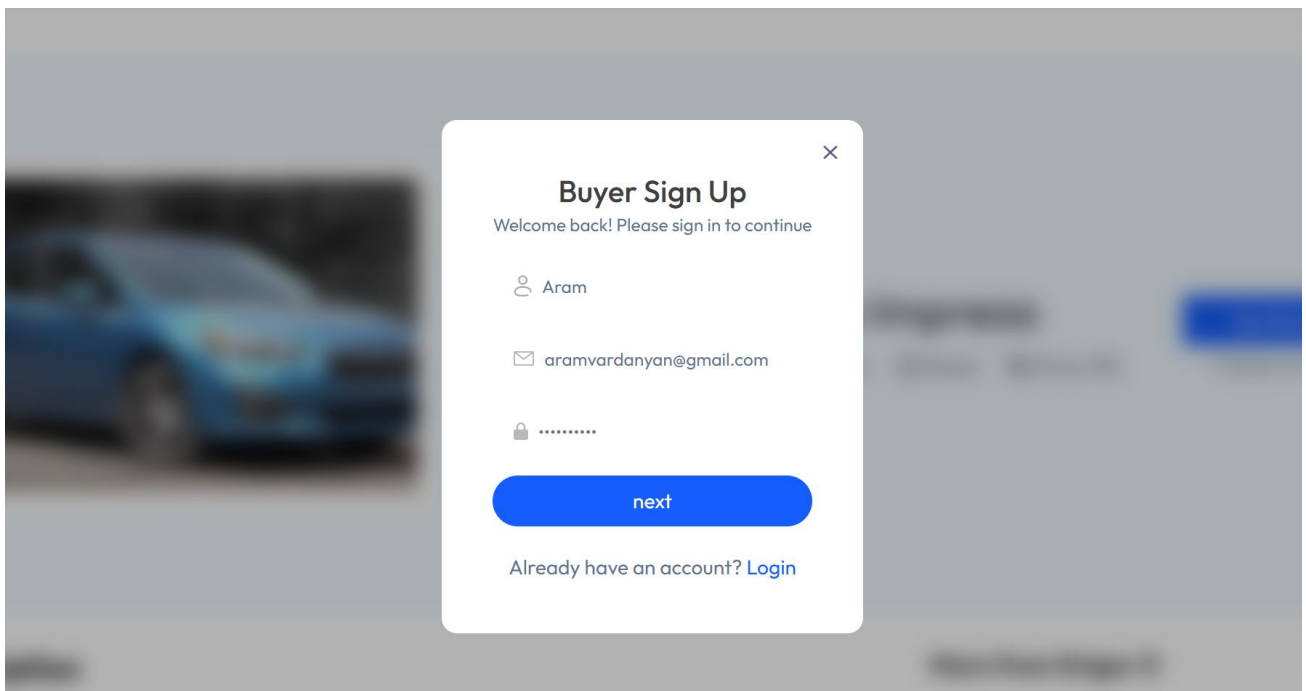
Կայքն ունի 2 տարբեր օգտատերերի դերային համակարգ՝ գնորդների և վաճառողների, հիմա ցույց կտրվի գնորդներին՝

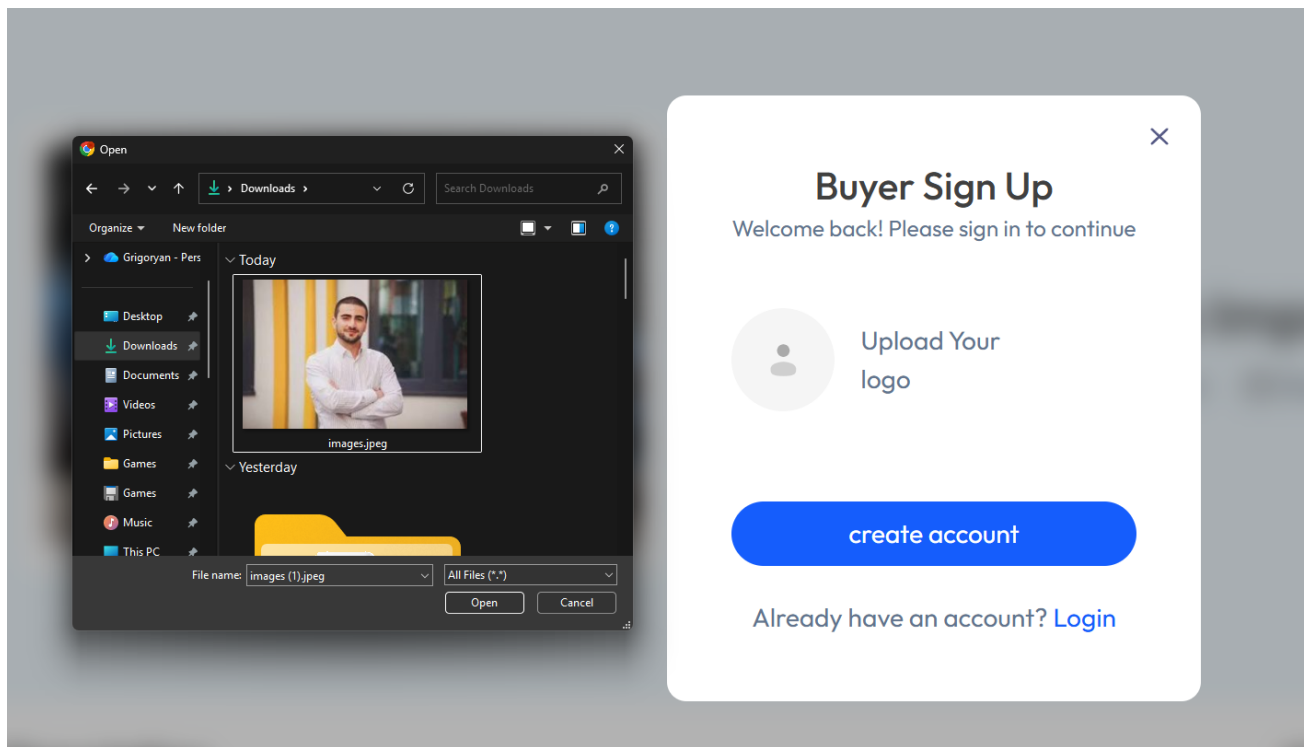
5.2.1 Գնորդ

Սեղմելով համապատասխան Login սեղմակը՝ կբացվի փոքր պատուհան, որտեղ մենք կարող ենք մուտք գործել, կամ բացել մեր գնորդի անձնական էջը;

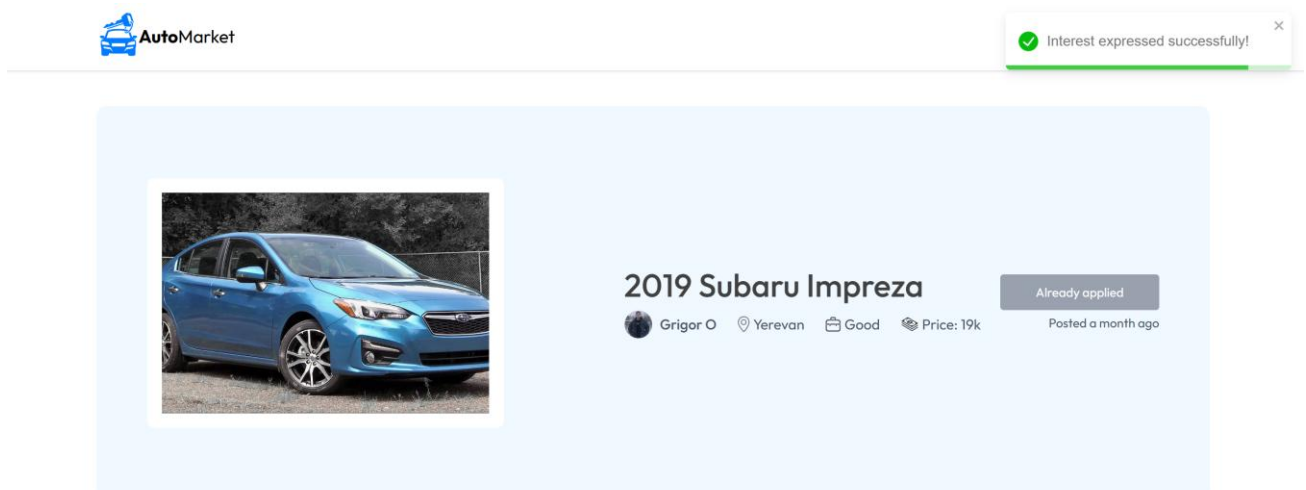


Սեղմելով մուտք գործելու սեղմակից ներքև՝ Sign Up կոճակի վրա, մենք կանցնենք մուտքագրման ռեժիմից անձնական էջի բացման ռեժիմ. ապա լրացնելով մեր անձնական տվյալները մենք կանցնենք հաջորդ փուլ:





Այս փուլում պետք է ընտրել նկար անձնական էջի համար: Սեղմելով նկարի ընտրման դաշտի վրա՝ կբացվի համակարգչի ֆայլային համակարգը և այնտեղից կարելի է ընտրել համապատասխան նկար և ապա սեղմելով էջի պատրաստման սեղմակի վրա՝ էջը պատրաստ կլինի:



Անձնական էջը պատրաստելուց հետո դուք կարելի է հարցումը հաջողությամբ ուղարկել, որի մասին կհուշի վերևում հայտնվող նամակը և սեղմակի փոփոխությունը:

My Car Inquiries

Seller	Car Model	Location	Inquiry Date	Status
 Grigor O	2019 Subaru Impreza	Yerevan	Apr 25, 2026	Pending

Մուտք գործելով գնորդի հարցումների էջ, (My Inquiries) կտեսնենք նրա ներկայիս հարցումները և ինֆորմացիա դրանց մասին՝ թե ում է ուղղված, ինչ ավտոմեքենայի վերաբերյալ է, ուղարկելու ժամը, պատասխանը և այլն:

5.2.2 Վաճառող

Վաճառողի մուտքը կամ էջի ստեղծումը հիմնականում տարբերվում է նրանով, որ երբ օգտատերը սեղմում է «Seller Login» կոճակի վրա և լրացնելուց հետո ուղարկում է հարցումը, նա տվյալների բազայում գրանցվում է որպես վաճառող: Այդ դեպքում նրա role դաշտում պահպանվում է *seller* արժեքը, մինչդեռ սովորական մուտքի (Login) դեպքում օգտատերը գրանցվում է որպես գնորդ՝ *buyer* դերակատարմամբ: Այս կերպ բոլոր օգտատերերը պահպանվում են նույն՝ Users խմբում, սակայն տարբերակվում են իրենց դերերով, այլ ոչ թե առանձին խմբերի բաժանմամբ:

Վաճառողի էջ մուտք գործելուց հետո առաջինը բացվում է Dashboard էջը, որտեղ ներկայացված է ընդհանուր տեղեկատվությունը և կառավարման գործիքները: Էջի ձախ հատվածում տեղադրված են երեք ենթաէջերի անցման կոճակներ, որոնք հնարավորություն են տալիս արագ նավարկելու տարբեր բաժինների միջև:

- Add Your Cars - Ավտոմեքենայի հայտարարություն ներմուծելու համար
- Manage Cars – Ավտոմեքենայի հայտարարություն տեղադրելու համար
- View Applications – Գնորդներից եկած հարցումները տեսնելու և պատասխանելու համար

Welcome, Grigor O

Add Your Car

Manage Cars

View Applications

Title

Type here

Car Image

Choose File No file chosen

Description

Normal

B I U

Category

Sedan

Location

Yerevan

Condition

New

Price

0

Հայտարարություններ ավելացնելու էջում կան բազմաթիվ դաշտեր, և նկար ավելացնելու տարածք: Համապատասխան դաշտերը ճիշտ լրացնելուց հետո սեղմելով ներքևում գտնվող կոճակի վրա հայտարարությունը կտեղադրվի:

Տեղադրված հայտարարությունները դիտելու և փոփոխություններ կատարելու համար, պետք է մուտք գործել Manage Cars էջ:

Welcome, Grigor O

Add Your Car

Manage Cars

View Applications

#	Title	Date	Location	Applicants	Visible
1	2019 Honda Civic Type R	Mar 20, 2026	Yerevan	1	☒
2	2019 Subaru Impreza	Mar 20, 2026	Yerevan	2	☐
3	2012 Lexus RX 350	Mar 20, 2026	Vanadzor	0	☐
4	2023 Kia EV6	Mar 20, 2026	Kapan	1	☒
5	2016 Nissan Leaf	Mar 20, 2026	Yerevan	0	☒
6	2022 Hyundai Tucson	Mar 19, 2026	Yerevan	1	☐
7	2017 BMW 530i	Mar 19, 2026	Vanadzor	0	☒
8	2019 Toyota Camry SE	Mar 19, 2026	Gyumri	0	☐
9	2018 Mercedes-Benz C300	Mar 19, 2026	Yerevan	0	☒
10	2020 BMW X5 xDrive40i	Mar 19, 2026	Yerevan	1	☒

Add new car

Այստեղ ցույց կտրվեն վաճառողի տեղադրած հայտարարությունները, տեղադրման ամսաթիվը, դիմորդների քանակը, և կարելի է փոխել տեսանելիությունը՝ Visible դաշտում եթե անջատենք տեսանելիությունը, հայտարարությունը չի ցուցադրվի հարթակում:

Add Your Car

Manage Cars

View Applications

#	User name	Car Title	Location	Action
1	 Aram	2019 Subaru Impreza	Yerevan	<button>Accept</button>
2	 David	2023 Kia EV6	Kapan	<button>Reject</button>
3	 Edgar Harutyunyan	2019 Subaru Impreza	Yerevan	...
4	 Daniel F	2019 Honda Civic Type R	Yerevan	...
5	 Edgar Harutyunyan	2022 Hyundai Tucson	Yerevan	Accepted
6	 Edgar Harutyunyan	2020 BMW X5 xDrive40i	Yerevan	Rejected

Հայտարարությունների դիմորդներին տեսնելու և պատասխանելու համար պետք է մուտք գործել View Applications ենթաէջ, որտեղ ցույց կտրվեն գնորդները, ովքեր հետաքրքրված են տեղադրված հայտարարություններով: Aram օգտատիրոջ կողմից ուղարկվել էր հարցում 5.2.1 վերնագրում, հարցումը վաճառողը կարող է ընդունել, կամ մերժել համապատասխան տողի Action հատվածում:

5.3 Օգտատիրոջ նույնականացման տեխնիկական գործընթացը (Login Flow)

Համակարգի աշխատանքը պատկերացնելու համար դիտարկենք օգտատիրոջ նույնականացման (Authentication) իրական պրոցեսը՝ սկսած ինտերֆեյսի հետ փոխազդեցությունից մինչև տվյալների բազայում կատարվող գործողությունները:

5.3.1. Հաճախորդային հարցման ձևավորում

Գործընթացը սկսվում է, երբ օգտատերը բացում է հավելվածի նույնականացման էջը և մուտքագրում իր տվյալները (էլ. փոստ և գաղտնաբառ):

- Նախքան հարցումն ուղարկելը, Frontend-ում (React/TypeScript) ստուգվում է տվյալների ձևաչափի ճշտությունը (օրինակ՝ էլ. փոստի վալիդացիա):
- «Մուտք» կոճակը սեղմելիս բրաուզերն ուղարկում է HTTPS POST հարցում որևէ սերվերային հասցե, օրինակ՝ <https://api.yourapp.com/v1/auth/login>: Հարցման մարմնում (Request Body) տվյալները փոխանցվում են JSON ձևաչափով:

5.3.2. Սերվերային մշակում

Node.js միջավայրում գործող սերվերը ստանում է հարցումը և իրականացնում հետևյալ քայլերը.

- Սերվերը գտնում է մուտքային տվյալները՝ SQL/NoSQL Injection տիպի հարձակումներից խուսափելու համար:

- Սերվերը հարցում է կատարում MongoDB-ին՝ համապատասխան էլ. փոստով օգտատեր փնտրելու համար:
 - *Տրամաբանական հարցումը:* `db.users.findOne({ email: "user@example.com" })`
- Քանի որ գաղտնաբառերը բազայում երբեք չեն պահվում բաց տեքստով, սերվերը կիրառում է հեշավորման ալգորիթմ (օրինակ՝ `bcrypt`): Մուտքագրված գաղտնաբառը հեշավորվում է և համեմատվում բազայում առկա արժեքի հետ:

5.3.3. Տոկենի թողարկում և Պատասխան (Response & JWT)

Եթե տվյալները համընկնում են, սերվերը գեներացնում է JWT (JSON Web Token):

- Տոկենը ներառում է օգտատիրոջ նույնականացուցիչը (ID) և նրա դերը (Վաճառող կամ Գնորդ), որը ստորագրված է սերվերային գաղտնի բանալիով (Secret Key):
- Սերվերը վերադարձնում է HTTP 200 OK կարգավիճակը և JSON օբյեկտը, որը պարունակում է տոկենը և օգտատիրոջ հիմնական տվյալները:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": { "role": "seller", "name": "Aram" }
}
```

5.3.4. Վերջնական փուլ Frontend-ում

- React հավելվածը ստանում է տոկենը և այն պահպանում է բրաուզերի հիշողության մեջ (LocalStorage կամ HTTP-only Cookies)՝ հետագա հարցումները վավերացնելու համար:
- Հիմնվելով օգտատիրոջ դերի վրա (օրինակ՝ `role: "seller"`), հավելվածը ավտոմատ կերպով ուղղորդում է օգտատիրոջը դեպի համապատասխան կառավարման վահանակ (Dashboard):

Եզրակացություն

Ամփոփելով ամբողջ աշխատանքը՝ կարելի է ասել, որ ստեղծված ավտոմեքենաների պլատֆորմը նպատակ ունի լուծելու իրական խնդիր, որը առկա է ներկայումս շուկայում՝ տեղեկատվության ոչ կենտրոնացվածություն, վստահության պակաս և գործարքների բարդություն: Հայաստանում նման կայքերի զարգացման սահմանափակ լինելը հաճախ պայմանավորված է ոչ միայն տեխնիկական, այլ նաև կազմակերպչական և վստահության գործոններով, ինչը դարձնում է այսպիսի համակարգերի անհրաժեշտությունը ավելի ակնհայտ:

Այս աշխատանքի ընթացքում ներկայացվել է համակարգի ամբողջական գաղափարը՝ սկսած դրա ստեղծման պատճառներից մինչև տեխնիկական իրականացում: Առանձնահատուկ ուշադրություն է դարձվել թե՛ ֆրոնտենդ, թե՛ բեքենդ հատվածներին՝ ցույց տալով, որ ժամանակակից վեբ հավելվածը պահանջում է երկու կողմերի համադրված և ճիշտ կառուցված աշխատանք: Ֆրոնտենդը ապահովում է օգտագործողի հետ հարմար և ինտուիտիվ փոխազդեցություն, մինչդեռ բեքենդը պատասխանատու է տվյալների մշակման, պահպանման և բիզնես տրամաբանության իրականացման համար:

Ներկայացված են նաև համակարգի հիմնական բաղադրիչները՝ օգտատերերի դերային մոդելը, ավտոմեքենաների հայտարարությունների կառուցվածքը և գործարքների համակարգը, որոնք միասին ապահովում են տվյալների ճիշտ կազմակերպում և կապակցվածություն: Բացի այդ, քննարկվել են նաև անվտանգության կարևոր մեխանիզմները, ինչպիսիք են գաղտնաբառերի հեշավորումը և սալթինգը, որոնք ապահովում են օգտատերերի տվյալների պաշտպանությունը:

Ընդհանուր առմամբ, այս նախագիծը ցույց է տալիս, թե ինչպես կարելի է ստեղծել լիարժեք, ընդլայնվող և անվտանգ ավտոմեքենաների առցանց պլատֆորմ՝ օգտագործելով ժամանակակից տեխնոլոգիաներ և ճիշտ նախագծային մոտեցումներ: Այն ոչ միայն տեխնիկական լուծում է, այլ նաև փորձ է՝ բարելավելու ավտոմեքենաների առքուվաճառքի գործընթացը՝ դարձնելով այն ավելի պարզ, արագ և վստահելի:

Օգտագործված Գրականություն

1. Mozilla Developer Network (MDN). JavaScript Documentation.
2. Mozilla Developer Network (MDN). HTML & CSS Reference.
3. React Documentation. React Official Guide. <https://react.dev>
4. TypeScript Documentation. TypeScript Handbook. <https://www.typescriptlang.org/docs/>
5. Node.js Documentation. Node.js Official API Reference. <https://nodejs.org>
6. Express.js Documentation. Express Framework Guide. <https://expressjs.com>
7. MongoDB Documentation. MongoDB Manual. <https://www.mongodb.com/docs>
8. Mongoose Documentation. MongoDB Object Modeling Tool Guide.
<https://mongoosejs.com/docs>
9. OWASP Foundation. OWASP Top 10 Web Application Security Risks. <https://owasp.org>
10. bcrypt.js / bcrypt Documentation. Password Hashing Guide.
11. REST API Design Guidelines. Microsoft REST API Best Practices.
12. Git Documentation. Version Control System Reference. <https://git-scm.com/doc>
13. Docker Documentation. Containerization Guide. <https://docs.docker.com>
14. JWT.io. JSON Web Token Introduction and Best Practices. <https://jwt.io>
15. Google Web Fundamentals. Modern Web Application Architecture. <https://web.dev>
16. Stack Overflow Documentation & Developer Discussions. <https://stackoverflow.com>

ԲՈՎԱՆԴԱԿՈՒԹՅՈՒՆ

Ներածություն

Գլուխ 1: Տեխնոլոգիաներ և ֆունկցիոնալություն

- 1.1: Օգտագործված տեխնոլոգիաները
- 1.2: Հիմնական ֆունկցիոնալ հնարավորությունները

Գլուխ 2: Տվյալների բազաներ

- 2.1: Տվյալների բազաների կառավարման համակարգեր (DBMS)
- 2.2: SQL (Ռեյադիոն տվյալների բազաներ)
- 2.3: NoSQL (Ոչ ռեյադիոն տվյալների բազաներ)
- 2.4: Ինչու է նախագծի համար ընտրվել NoSQL (MongoDB)

Գլուխ 3: Վեբ-համակարգերի ճարտարապետական հիմունքները

- 3.1: Վեբ-բրաուզեր և հաճախորդային մակարդակ
- 3.2: HTML (HyperText Markup Language)
- 3.3: CSS (Cascading Style Sheets)
- 3.4: JavaScript

Գլուխ 4: Վեբ-սերվեր և սերվերային մակարդակ

- 4.1: Սերվեր
- 4.2: Node.js

Գլուխ 5: Կառուցվածքը և ինչպես օգտվել կայքից

- 5.1: Բազայի կառուցվածքը
- 5.2: Ինչպես օգտվել կայքից
- 5.3: Օգտատիրոջ նույնականացման տեխնիկական գործընթացը (Login Flow) Եզրակացություն